Day 73 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 4 reference

Watch: Bahdanau vs Luong Attention — Additive vs Multiplicative (SourceCode AI) | Bahdanau Attention Vs Luong Attention (CampusX)

`day73_dot_product_attention.py`, segment by segment, in the order `main()` uses things. The LSTM, data and Day 71 attention code is reused verbatim from Days 63–72 and explained there; this reference covers what is new. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: s = decoder state (the query) · h_i = encoder state i (key and value) · e_i = score · α = attention weights · W_g = the general score's matrix · m = batch, n = encoder steps, h = hidden size (in einsum strings)

Segment 0 — Setup

Settings

CODE

```
SEEDS = [0, 1, 2]
EVAL_LENGTHS = (4, 8, 12, 16, 20)
STEPS = 20000
EVERY = 2500
SCORES = ("additive", "dot", "general")
COLORS = ("#7f7f7f", "#d62728", "#1f77b4")
```

- `SCORES = ("additive", "dot", "general")` — a tuple of names; every experiment loops over it.
- `COLORS` — one colour per score, in the same order, so a score has the same colour in every plot.

Segment 1 — The two new scores

dot_scores and its backward pass

MATH

$$e_i = s \cdot h_i \;\Rightarrow\; \partial L/\partial s = \Sigma_i\, de_i\, h_i, \quad \partial L/\partial h_i = de_i\, s$$

CODE

```
def dot_scores(s_prev, Hs):
    return np.einsum("mh,mnh->mn", s_prev, Hs)
def dot_scores_backward(de, s_prev, Hs):
    ds = np.einsum("mn,mnh->mh", de, Hs)
    dHs = de[:, :, None] * s_prev[:, None, :]
    return ds, dHs
```

- `np.einsum("mh,mnh->mn", s_prev, Hs)` — for each example m and step n, sum over h of s[m,h] times Hs[m,n,h]: one dot product per encoder state.
- `np.einsum("mn,mnh->mh", de, Hs)` — the reverse: sum over n of de[m,n] times Hs[m,n,:].
- `de[:, :, None] * s_prev[:, None, :]` — None adds an axis of size 1, so (m, n, 1) times (m, 1, h) broadcasts to (m, n, h).

general_scores and its backward pass

MATH

$$q = s\,W_g,\quad e_i = q \cdot h_i \;\Rightarrow\; dq = \Sigma_i\, de_i\, h_i,\quad ds = dq\, W_g^{\,T},\quad dW_g = s^T\, dq$$

CODE

```
def general_scores(s_prev, Hs, Wg):
    q = s_prev @ Wg
    return np.einsum("mh,mnh->mn", q, Hs), q
def general_scores_backward(de, s_prev, Hs, q, Wg):
    dq = np.einsum("mn,mnh->mh", de, Hs)
    dHs = de[:, :, None] * q[:, None, :]
    return dq @ Wg.T, dHs, s_prev.T @ dq
```

- `return ..., q` — the forward pass returns two things; the projected query is kept for the backward pass.
- `dq @ Wg.T` — back through q = s Wg to s.
- `s_prev.T @ dq` — the matrix's gradient: (h, m) times (m, h) sums over the batch.

Segment 2 — The model with the score as a setting

ScoredAttnSeq2Seq.__init__

CODE

```
class ScoredAttnSeq2Seq(AttnSeq2Seq):
    def __init__(self, score="additive", hidden=32, attn=32, seed=0, forget_bias=1.0, lr=0.5,
        clip=1.0):
        self.score = score
        if score == "additive":
            super().__init__(hidden=hidden, attn=attn, seed=seed, forget_bias=forget_bias, lr=lr,
                clip=clip)
            return
        rng = np.random.RandomState(seed)
        H = hidden
        self.H, self.lr, self.clip, self.init_from_encoder = H, lr, clip, True
        self.P = {"We": rng.randn(V + H, 4 * H) / np.sqrt(V + H), "be": np.zeros(4 * H),
                  "Wd": rng.randn(V + 2 * H, 4 * H) / np.sqrt(V + 2 * H), "bd": np.zeros(4 * H)}
        if score == "general":
            self.P["Wg"] = rng.randn(H, H) / np.sqrt(H)
        self.P.update(Why=rng.randn(2 * H, V) / np.sqrt(2 * H), by=np.zeros(V))
        self.P["be"][:H] = forget_bias; self.P["bd"][:H] = forget_bias
```

- `class ScoredAttnSeq2Seq(AttnSeq2Seq)` — inheritance: the new class has every method of Day 71's class unless it redefines it.
- `super().__init__(...); return` — for the additive score, run Day 71's constructor and stop: the same random draws, so the same model.
- `rng.randn(V + 2 * H, 4 * H)` — the decoder input is the token plus the context (H numbers), and its own state.
- `self.P.update(Why=..., by=...)` — adds two entries to the dictionary at once.

encode, scores and step

MATH

$$\alpha = softmax(e), \quad context = \Sigma_i \alpha_i h_i$$

CODE

```
def encode(self, Xs):
    if self.score == "additive":
        return super().encode(Xs)
    m = Xs.shape[0]
    z = np.zeros((m, self.H))
    Hs, Cs, gates = lstm_forward(Xs, z, z, self.P["We"], self.P["be"])
    return Hs, Cs, gates, Hs, Hs[:, -1], Cs[:, -1]
def scores(self, s, Hs, keys):
    if self.score == "additive":
        return additive_scores(s, keys, self.P["Wa"], self.P["va"])
    if self.score == "dot":
        return dot_scores(s, Hs), None
    return general_scores(s, Hs, self.P["Wg"])
def step(self, y_onehot, s, c, Hs, keys):
    e, cache = self.scores(s, Hs, keys)
    ctx, alpha = soft_read(Hs, e, np.ones(e.shape, bool))
    x = np.concatenate([y_onehot, ctx], axis=1)
    s_new, c_new, gates = lstm_step_forward(x, s, c, self.P["Wd"], self.P["bd"])
    out = np.concatenate([s_new, ctx], axis=1)
    return s_new, c_new, dict(s_prev=s, c_prev=c, t=cache, alpha=alpha, ctx=ctx, x=x, s=s_new,
        c=c_new, gates=gates, out=out)
```

- `return Hs, Cs, gates, Hs, ...` — the fourth value is the keys: for the multiplicative scores they are the states themselves.
- `return dot_scores(s, Hs), None` — a tuple: the scores and an empty cache, so all three branches return the same shape of result.
- `np.ones(e.shape, bool)` — a mask that keeps every slot: all sequences in a batch have the same length.
- `dict(s_prev=s, ...)` — every value the backward pass will need, kept under a name.

loss_and_grads: the backward loop's new lines

MATH

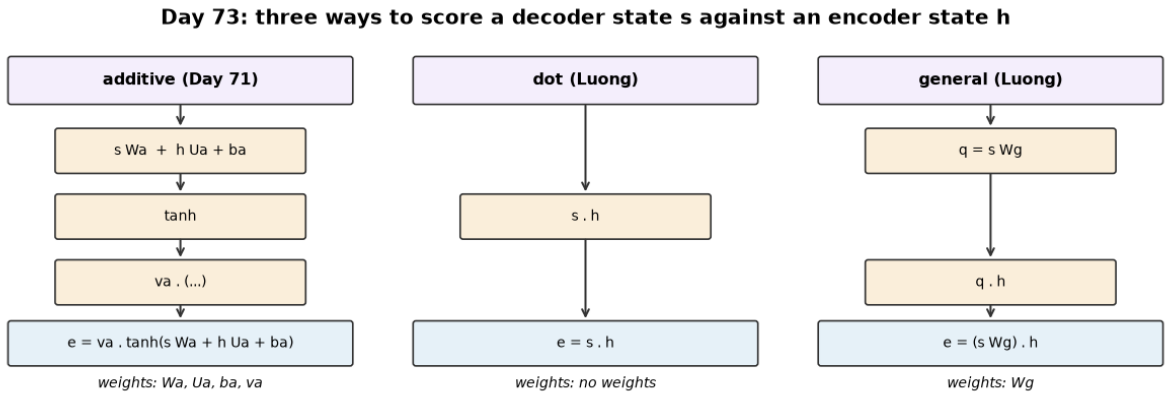
$$dh_i = \alpha_i \, d\,context \;\; (values) \;\; + \;\; de_i \, s \; or \; de_i \, q \;\; (keys)$$

CODE

```
dHs += alpha[:, :, None] * dctx[:, None, :]
dalpha = np.einsum("mh,mnh->mn", dctx, Hs)
de = alpha * (dalpha - (alpha * dalpha).sum(axis=1, keepdims=True))
if self.score == "dot":
    dsp, dH_score = dot_scores_backward(de, st["s_prev"], Hs)
else:
    dsp, dH_score, dWg = general_scores_backward(de, st["s_prev"], Hs, st["t"], P["Wg"])
    g["Wg"] += dWg
dHs += dH_score
ds, dc = ds_prev + dsp, dc_prev
```

- `dHs += alpha[:, :, None] * dctx[:, None, :]` — the memory as the values of the weighted sum (Day 71).
- `alpha * (dalpha - (alpha * dalpha).sum(...))` — the softmax's backward pass, row by row.
- `dHs += dH_score` — NEW: the memory is also the keys, so it gets a second gradient through the scores.
- `ds_prev + dsp` — the previous decoder state fed the LSTM step and the scores: both gradients add.

SYSTEM DIAGRAM



The three scores; everything else in the model is Day 71's.

Segment 3 — Experiments

train and train_cached

CODE

```
def train(score, task, hidden, seed, steps=STEPS, lo=3, hi=20, every=EVERY):
    net = ScoredAttnSeq2Seq(score, hidden=hidden, seed=seed)
    rng = np.random.RandomState(seed + 100)
    tests = {n: fixed_test(n, task) for n in EVAL_LENGTHS}
    curve = []
    for step in range(1, steps + 1):
        n = rng.randint(lo, hi + 1)
        Xs, Xd, Y, _ = make_batch(rng, 32, n, task)
        net.train_step(Xs, Xd, Y)
        if step % every == 0:
            curve.append([exact_and_token(net, tests[n])[0] for n in EVAL_LENGTHS])
    return net, np.array(curve)
RUNS = {}
def train_cached(*args):
    if args not in RUNS:
        RUNS[args] = train(*args)
    return RUNS[args]
```

- `np.random.RandomState(seed + 100)` — the data stream depends only on the seed, so every score sees the same batches.
- `np.array(curve)` — a list of lists becomes a (checkpoints, lengths) array.
- `def train_cached(*args)` — *args collects all the positional arguments into one tuple, used as a dictionary key.
- `RUNS[args]` — Sections 3 to 7 ask for the same models; each trains once.

alignment: Day 72's three numbers

MATH

$$entropy = -\Sigma_i \alpha_i \log \alpha_i \quad (\log 20 = 3.00 \text{ means spread evenly})$$

CODE

```
def alignment(net, task, n):
    Xs, _, Y, _ = fixed_test(n, task)
    pred, W = net.generate(Xs, n + 1, return_weights=True)
    W = W[:, :n]
    j = np.arange(n)
    target = j if task == "copy" else n - 1 - j
    entropy = -(W * np.log(W + 1e-12)).sum(axis=2)
    return (W.argmax(axis=2) == target).mean(), W[:, j, target].mean(), entropy.mean()
```

- `target = j if task == "copy" else n - 1 - j` — a conditional expression: the right input position for each output.
- `W[:, j, target]` — two index arrays of the same length pick one entry per output step: (j[0], target[0]), (j[1], target[1]), ...
- `W.argmax(axis=2) == target` — a (m, n) array of True or False; its mean is the share of True.

demo_training: when every seed is perfect

CODE

```
for sc in SCORES:
    runs = [train_cached(sc, "copy", 32, s)[1] for s in SEEDS]
    done = [i for i in range(len(runs[0])) if all(np.all(r[i] == 1.0) for r in runs)]
    first = f"{(done[0] + 1) * EVERY} steps" if done else "not within the budget"
    print(f"  {sc:<9}" + "  ".join(f"seed {s} {r[-1][j20]:.3f}" for s, r in zip(SEEDS, runs)) + f"
        all perfect from: {first}")
```

- `[i for i in range(...) if all(...)]` — a list comprehension with a condition: the checkpoints where every seed is perfect.
- `np.all(r[i] == 1.0)` — True only if every length is at exactly 1.000.
- `... if done else "not within the budget"` — an empty list counts as False.
- `zip(SEEDS, runs)` — walks the seeds and their curves in step.

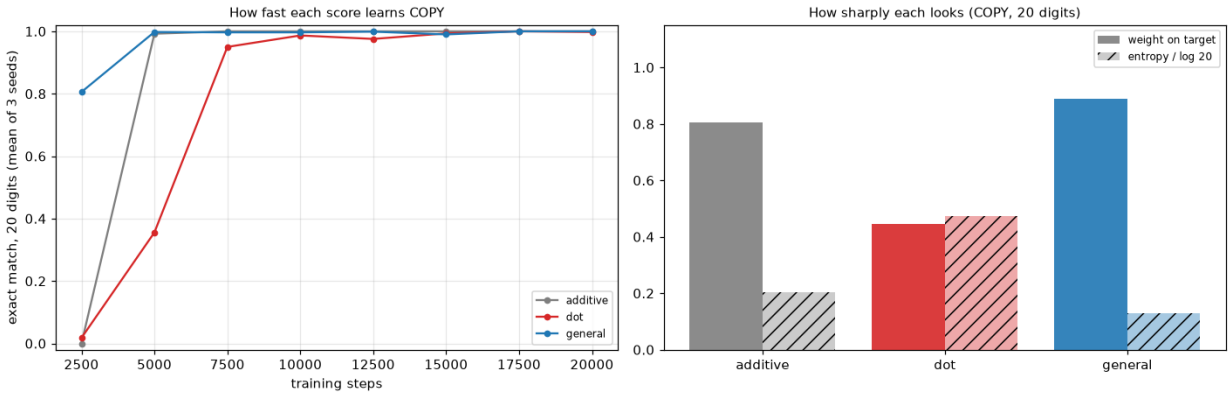
demo_cost: timing with a default argument

CODE

```
t = {sc: best(lambda net=ScoredAttnSeq2Seq(sc, hidden=32): net.loss_and_grads(Xs, Xd, Y)) for sc
    in SCORES}
print(f"\n{'score':<10}{'ms per step':>13}{'vs additive':>13}{'score weights':>15}")
for sc in SCORES:
    n_score = sum(ScoredAttnSeq2Seq(sc, hidden=32).P[k].size for k in ["Wa", "Ua", "ba", "va",
        "Wg"]
                  if k in ScoredAttnSeq2Seq(sc, hidden=32).P)
```

- `lambda net=...: ...` — the default value is built once, when the lambda is made: the timing does not include building the model.
- `best(fn)` — the fastest of 5 timings of 50 calls; only the ratios within a run are reported.
- `for k in [...] if k in ...P` — counts only the weight matrices this score actually has.

REAL GRAPH



Real graph from the script: learning speed, and how sharply each score looks.

Quick Syntax Glossary — new or reinforced constructs

- `class B(A): ... super().f()` — inherit from A, and call A's version of a method B redefines.
- `np.einsum("mh,mnh->mn", a, b)` — a sum of products described by index letters; letters missing on the right are summed.
- `x[:, :, None]` — add an axis of size 1 for broadcasting.
- `def f(*args)` — collect positional arguments into a tuple.
- `lambda x=value: ...` — a default argument, evaluated once when the lambda is made.
- `a if cond else b` — a conditional expression.
- `A[:, rows, cols]` — pick paired entries with two index arrays.

End of Day 73 syntax reference. For the experiments and what they showed, see the Day 73 complete guide.